

میدان IOAI

- محدودیت زمان: 5 minutes
- فضای ذخیره‌سازی: GB 5
- اندازه (حجم) رامل: solution.ipynb, custom_model.py در مجموع ≥ 1 MB
- مدل‌های از پیش آموزش‌دیده: هیچ‌چیز! — آموزش همه چیز از ابتدا است، بدون اینترنت هنگام نمردهی
- امتیاز مبنا: 31.2187

مسئله

شهردار آستانه می‌خواهد شهر را با لوگوهای سبکدار شده مختص IOAI تزئین کند. لذا در هر لوگو، عبارت انگلیسی IOAI به همراه یک پس‌زمینه (background) وجود دارد که اصطلاحاً سبکدار (stylized) شده است. او به‌عنوان یک آماردان، همه‌چیز — از جمله لوگو — را یک تابع فضایی (spatial function) به صورت کلی $F(x, y, \overline{W})$ در نظر می‌گیرد، که در آن $x, y \in [0, 1]$ مختصات روی یک صفحه 2D را نشان می‌دهند و $\overline{W}$ مجموعه‌ای از پارامترهای پنهان است که ویژگی‌های سبکی، مانند رنگ‌ها و زاویه‌های حروف، را تعریف می‌کند.

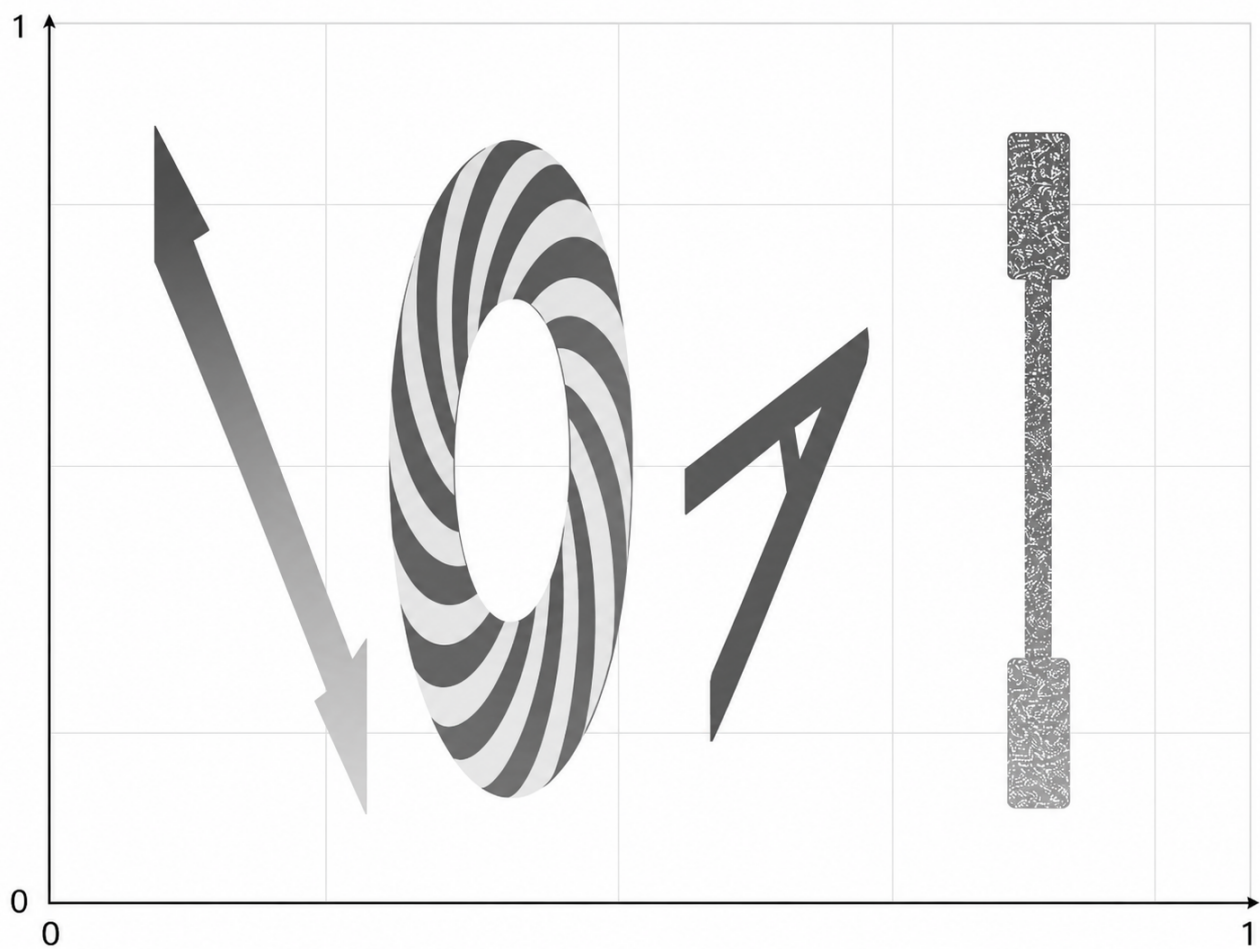
از آنجا که F بیش از حد پیچیده است و نمی‌توان آن را به‌صورت یک معادله ریاضی صریح بیان کرد، وظیفه شما آموزش یک شبکه عصبی (neural network) برای تقریب آن است. شبکه برای هر زوج مختصات (x, y) یک مقدار میدان IOAI یا IOAI Field خروجی خواهد داد و یک نمایش کامل نقشه حرارتی (heatmap) از لوگو را در سراسر صفحه تولید خواهد کرد. در ادامه، نمونه‌ای از نمایش نقشه حرارتی F با چند پارامتر پنهان مشخص $\overline{W}$ آمده است.

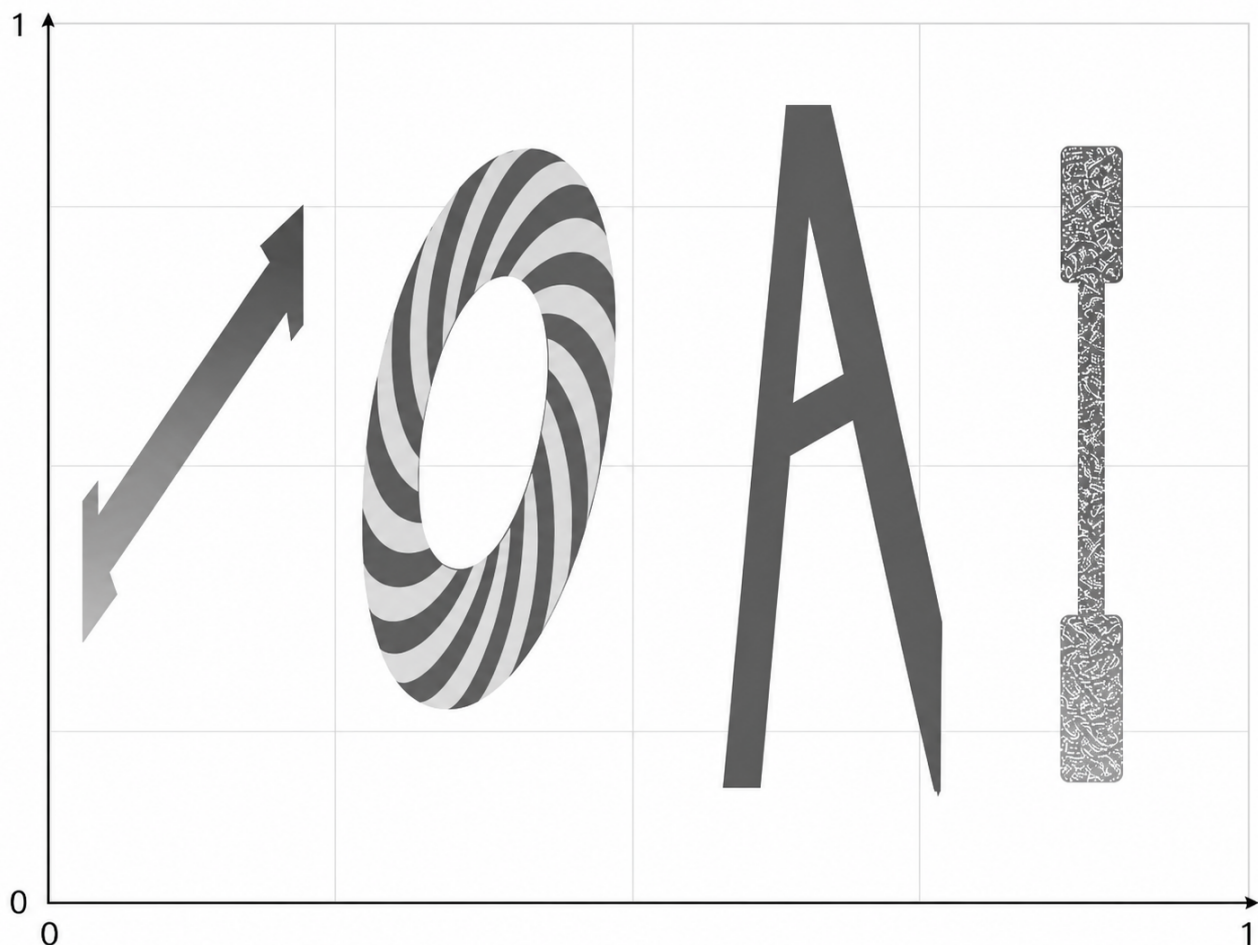


میدان IOAI از چه چیزهایی تشکیل شده است؟ چهار حرف و یک پس‌زمینه.

- مقادیر درون حرف نخست I بسیار بزرگ‌اند ($1e+10$ و بیشتر) و دارای گرادیان خطی هستند (به صورت خطی بزرگ می‌شوند)
- مقادیر در حرف o الگویی مارپیچی (spiral) را نشان می‌دهند
- مقدار درون حرف A همیشه 1 است
- مقادیر درون حرف آخر I باید مقادیری تصادفی از بازه $[-2026, 2026]$ باشند، حتی اگر یک نقطه دو بار ارزیابی شود
- بیرون از حروف (که می‌شود پس‌زمینه)، مقدار همیشه صفر است

تابع دارای پارامترهای پنهان $\overline{W}$ است که بر مقیاس و شیب حروف و نیز بازه مقادیر درون حرف نخست I تأثیر می‌گذارد. با این حال، حروف با یکدیگر تلاقی نخواهند داشت. در ادامه، چند نمونه تصویری از شکل میدان IOAI با مقادیر متفاوت $\overline{W}$ آمده است:





آنچه در اختیار شما قرار داده شده است:

- لطفاً این بخش را به انگلیسی نیز مطالعه بفرمایید.

این مسئله هیچ dataset ندارد. در عوض، تابع مولدی (generator function) در اختیار شما قرار گرفته است که با فایل `config` از نوع JSON در `data/train_config/field_config.json` پی‌کربندی (`configure`) می‌شود. این فایل کانفیگ فعلی در زمان تمرین (`train`) به شما داده شده است، ولی در زمان تست تغییر خواهد کرد.

به عبارت دیگر، `config` آزمون (یعنی کانفیگ زمان تست) پنهان است، اما ماهیتی مشابه دارد. وظیفه شما این است که با استفاده از هر مقدار داده‌ای که می‌خواهید، روی مولد داده‌شده با کانفیگ فعلی برآش انجام دهید. دقت کنید توزیع‌های «آموزش» و «آزمون» شما از یک مولد یکسان تولید می‌شوند — فقط نمی‌دانید روی کدام نقاط (x_i, y_i) ارزیابی خواهید شد.

ارسال شما باید شامل موارد زیر باشد:

- کلاس مدل آموزش که با نام `custom_model.py` ذخیره شده است. این مدل باید از کلاس `torch.nn.Module` ارث‌بری کند و فقط از `import torch` استفاده کند. این فایل باید شامل کلاس `CustomModel` باشد که در نوت‌بوک `solution.ipynb` استفاده می‌شود.
- نوت‌بوک `solution.ipynb` که وزن‌های `model.pt` را تولید خواهد کرد

امتیازدهی

برای هر ناحیه، حداقل امتیاز 0 و حداکثر امتیاز 1 است. امتیاز نهایی روی هر پنج ناحیه (چهار ناحیه برای هر یک از حروف و یک ناحیه برای پس‌زمینه) میانگین‌گیری و در 100 ضرب می‌شود. یک **جریمه پارامتر** وجود دارد:

اگر مدل شما بیش از **20260** پارامتر داشته باشد، امتیاز نصف می‌شود.

تعداد پارامتر با `sum(p.numel() for p in model.parameters())` اندازه‌گیری می‌شود. انتظار داریم مدل شما در حالت تصادفی نیز عمل کند و `nn.Dropout` متعلق به `PyTorch` بخشی از مدل باشد.

برای نواحی استاندارد

برای هر ناحیه R (حرف نخست I، حرف O، حرف A، و Background)، که شامل آخرین حرف I نمی‌شود، مدل را روی $N_R = 512$ نقطه آزمون (x_i, y_i) با مقادیر واقعی v_i و پیش‌بینی‌های $\hat{v}_i$ ارزیابی می‌کنیم. از میانگین قدر مطلق خطای نرمال‌شده (MAE) به‌عنوان معیار اصلی استفاده می‌کنیم. MAE به‌صورت زیر تعریف می‌شود:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

و نرمال‌سازی به‌صورت زیر انجام می‌شود:

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

که در آن $s_R > 0$ یک ثابت مقیاس است.

برای ناحیه حرف آخر I

در این ناحیه، **dropout** هنگام ارزیابی فعال است. برای هر نقطه آزمون j :

1. مدل را $K = 10$ بار اجرا می‌کنیم تا $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ به دست آید.
2. اگر هر خروجی خارج از بازه $[-2026, 2026]$ باشد، آنگاه $\text{pointScore}(j) = 0$.
3. در غیر این صورت، انحراف معیار σ_j از K خروجی را محاسبه و آن را به امتیاز تبدیل می‌کنیم:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

که در آن $s_E > 0$ یک ثابت مقیاس ثابت است.

امتیاز ناحیه برابر با میانگین روی تمام نقاط ناحیه است:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

که در آن $N_E = K * N_R$.

به بیان ساده، هرچه تنوع (diversity) بیشتری داشته باشید، امتیاز شما برای این ناحیه بیشتر خواهد بود. نمی‌توانید از تصادفی‌بودن به شکل خالص استفاده کنید؛ این ممنوعیت شامل توابع `rand*` و `_uniform` در **PyTorch** نیز می‌شود. تصادفی‌بودن باید از استنتاج (inference) با **dropout** فعال حاصل شود.

نحوه ارسال

1. `solution.ipynb` را باز کنید و تمام سلول‌ها را اجرا کنید.
2. مدل CustomModel را در `custom_model.py` بهبود دهید
3. مطمئن شوید که آخرین سلول شما مدل را در فایل `model.pt` ذخیره می‌کند.
4. در زبانه Git در JupyterLab، فایل‌های `solution.ipynb` و `custom_model.py` را `comment`، `stage` و `commit` کنید، سپس آن‌ها را `push` کنید.
5. به صفحه مسابقه بازگردید و روی **Submit** کلیک کنید. کامنت ارسال باید با کامنت مرحله قبل یکسان باشد.